

Notes on Linear Regression

Luis Mendez

Question: What is linear regression doing, and how does a neural network learn it?

Answer:

1. The Model

Linear regression fits a straight line through data: given input x , it predicts

$$\hat{y} = ax + b,$$

where a is the **slope** and b is the **intercept**. With D input features this generalizes to $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ — exactly a single neuron with *no* activation function (or, equivalently, a linear ϕ). Framing it this way, a dense layer of one unit with linear activation *is* linear regression.

2. Loss: Mean Squared Error

Training picks a, b to minimize the squared distance between predictions and targets, averaged over N examples:

$$C(a, b) = \frac{1}{N} \sum_{i=1}^N (y_i - (ax_i + b))^2.$$

Squaring penalizes large errors more than small ones and keeps C smooth and differentiable everywhere, which is what makes both routes below possible.

3. Two Ways to Solve It

- **Closed form (OLS):** because C is quadratic and convex in a, b , setting $\partial C / \partial a = \partial C / \partial b = 0$ and solving directly gives

$$a = \frac{\sum x_i y_i - \bar{y} \sum x_i}{\sum x_i^2 - \bar{x} \sum x_i}, \quad b = \bar{y} - a\bar{x}.$$

No iteration needed — one pass over the data.

- **Gradient descent:** treat a, b as the weight and bias of a one-neuron network and train it the same way as any other neuron (see *Notes on Neurons*): repeatedly nudge a, b opposite ∇C until the loss stops improving. Slower and approximate, but the only option once the model has more layers or nonlinearity, where no closed form exists.

Both routes should agree, up to numerical precision, on a purely linear problem — a good sanity check after training.

4. Linearizing an Exponential Trend

Not every relationship is linear on its own scale, but some become linear after a transform. Moore's Law states

that transistor count roughly doubles every two years, i.e. $y \approx y_0 \cdot 2^{x/2}$, an exponential in the raw data. Taking the log of both sides,

$$\log y = \log y_0 + \frac{\log 2}{2} x,$$

which is linear in x . So $\log y$ vs. x can be fit with ordinary linear regression, and the fitted slope a converts back into a doubling time via $a = \frac{\log 2}{2 \text{ years}} \Rightarrow \text{doubling time} = \log 2 / a$. Centering x (subtracting its mean) before fitting does not change a ; it only keeps b interpretable and improves numerical conditioning for gradient descent.

5. Keras Example

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(1),
])

model.compile(
    optimizer=tf.keras.optimizers.SGD(0.001, 0.9),
    loss='mse',
)

r = model.fit(X, Y, epochs=100)
```

A single `Dense(1)` layer with the default (linear) activation is the whole model: one weight, one bias. The learned weight and bias recovered from `model.layers[0].get_weights()` should match the closed-form a, b computed directly from the data.

6. Saving and Reloading

```
model.save('linearregression.h5')

loaded = tf.keras.models.load_model('linearregression.h5')
np.allclose(model.predict(X), loaded.predict(X))
```

Saving stores the architecture, learned weights, and optimizer state; reloading should reproduce identical predictions, which is a useful check that nothing was lost in the round trip.

7. Putting It Together

- Linear regression = one neuron, linear activation, MSE loss.
- With one input feature it can be solved exactly (OLS) or learned iteratively (gradient descent); both should agree.
- A log transform turns an exponential trend (e.g. Moore's Law) into a linear one, so the same tools apply.

- The same neuron-and-loss machinery scales up to deep networks once activations become nonlinear and the closed form disappears.